8.2.1 DeFi 팔렛 구현하기

개요

이 고급 튜토리얼에서는 실제 DeFi(탈중앙화 금융) 기능을 제공하는 팔렛을 구현합니다. 자동화된 마켓 메이커(AMM) 기반의 탈중앙화 거래소(DEX)를 구축하여 토큰 스왑, 유동성 제공, 수수료 분배 등의 핵심 DeFi 기능을 학습합니다.

학습 목표

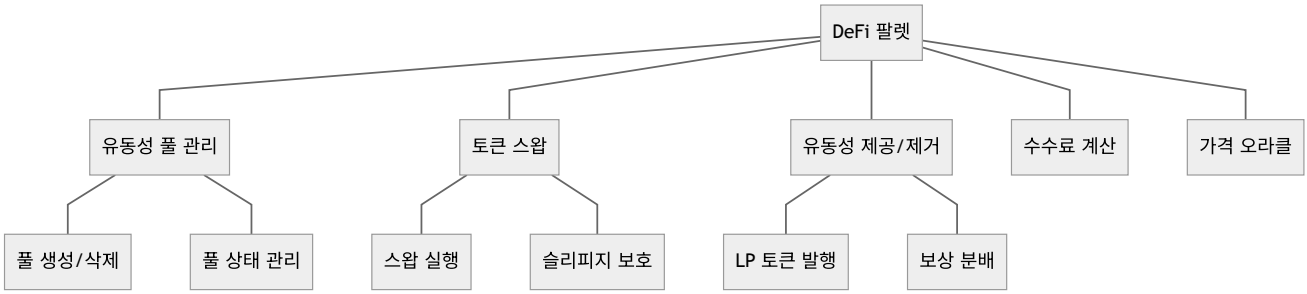
- AMM(Automated Market Maker) 알고리즘 구현
- 멀티 토큰 지원 및 유동성 풀 관리
- 슬리피지 보호 및 가격 계산 로직
- 유동성 공급자 보상 시스템
- 고급 오류 처리 및 보안 고려사항

사전 요구사항

- 기초 팔렛 개발 완료
- Rust 고급 문법 (제네릭, 트레이트, 매크로)
- DeFi 기본 개념 이해 (AMM, 유동성 풀, 슬리피지)
- 수학적 계산 (제곱근, 비율 계산)

1단계: DeFi 팔렛 아키텍처 설계

1.1 핵심 구성 요소



1.2 데이터 구조 설계

```
// 유동성 풀 정보
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct PoolInfo<AssetId, Balance> {
pub asset_a: AssetId,
pub asset_b: AssetId,
pub reserve_a: Balance,
pub reserve_b: Balance,
pub total_supply: Balance,
pub fee_rate: u32, // 기본 단위: 10000 = 100%
}

// 유동성 공급자 정보
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
```

```
pub struct LiquidityProvider<AccountId, Balance> {
pub account: AccountId,
pub lp_tokens: Balance,
pub rewards_debt: Balance,
}

// 스왑 경로
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct SwapPath<AssetId> {
pub path: Vec<AssetId>,
pub pools: Vec<u32>, // 풀 ID들
}
```

2단계: 기본 팔렛 구조 생성

2.1 프로젝트 설정

```
# 새 팔렛 디렉토리 생성
mkdir -p pallets/defi-dex/src
cd pallets/defi-dex
```

2.2 Cargo.toml 설정

```
[package]
```

```
name = "pallet-defi-dex"

version = "4.0.0-dev"

description = "DeFi DEX Pallet for Substrate"

authors = ["Your Name <your.email@example.com>"]

homepage = "https://substrate.io"

edition = "2021"

license = "MIT-0"

publish = false


[package.metadata.docs.rs]

targets = ["x86_64-unknown-linux-gnu"]


[dependencies]

codec = { package = "parity-scale-codec", version = "3.6.1", default-features =

scale-info = { version = "2.5.0", default-features = false, features = ["derive"

serde = { version = "1.0.188", default-features = false, features = ["derive", "


# Substrate dependencies

frame-benchmarking = { version = "4.0.0-dev", default-features = false, optional

frame-support = { version = "4.0.0-dev", default-features = false, git = "https:

frame-system = { version = "4.0.0-dev", default-features = false, git = "https:/

sp-runtime = { version = "24.0.0", default-features = false, git = "https://gith

sp-std = { version = "8.0.0", default-features = false, git = "https://github.co
```

```
# Math dependencies

sp-arithmetic = { version = "16.0.0", default-features = false, git = "https://g



[dev-dependencies]

sp-core = { version = "21.0.0", git = "https://github.com/paritytech/polkadot-sd

sp-io = { version = "23.0.0", git = "https://github.com/paritytech/polkadot-sdk.

pallet-balances = { version = "4.0.0-dev", git = "https://github.com/paritytech/

pallet-assets = { version = "4.0.0-dev", git = "https://github.com/paritytech/po



[features]

default = ["std"]

std = [

"codec/std",

"frame-benchmarking?/std",

"frame-support/std",

"frame-system/std",

"scale-info/std",

"serde/std",

"sp-runtime/std",

"sp-std/std",

"sp-arithmetic/std",

]
```

```
runtime-benchmarks = ["frame-benchmarking/runtime-benchmarks"]

try-runtime = ["frame-support/try-runtime"]
```

2.3 기본 팔렛 구조

```
// pallets/defi-dex/src/lib.rs

#![cfg_attr(not(feature = "std"), no_std)]


pub  use pallet::*;


use frame_support::{

dispatch::{DispatchResult, DispatchError},

pallet_prelude::*,

traits::{

fungibles::{Inspect, Mutate, Transfer},

tokens::{Preservation, Precision},

Get,

},

PalletId,

};

use frame_system::pallet_prelude::*;

use sp_arithmetic::{
```

```
traits::{Saturating, Zero, CheckedAdd, CheckedSub, CheckedMul, CheckedDiv},
Permill,
};
use sp_runtime::{
traits::{AccountIdConversion, Bounded},
ArithmeticError,
};
use sp_std::{vec::Vec, collections::btree_map::BTreeMap};


#[frame_support::pallet]
pub mod pallet {
use super::*;


/// 팔렛 설정 트레이트
#[pallet::config]
pub trait Config: frame_system::Config {
/// 런타임 이벤트 타입
type RuntimeEvent: From<Event<Self>> + IsType<<Self as frame_system::Config>::


/// 자산 타입 (토큰 ID)
type AssetId: Parameter + Copy + Ord + Zero;


/// 잔액 타입
```

```
type Balance: Parameter + Copy + Ord + Zero + Saturating + CheckedAdd + CheckedS


/// 자산 관리 트레이트
type Assets: Inspect<Self::AccountId, AssetId = Self::AssetId, Balance = Self::B
+ Mutate<Self::AccountId>
+ Transfer<Self::AccountId>;


/// 팔렛 ID (계정 생성용)
#[pallet::constant]
type PalletId: Get<PalletId>;


/// 기본 수수료율 (10000 = 100%)
#[pallet::constant]
type DefaultFeeRate: Get<u32>;


/// 최소 유동성
#[pallet::constant]
type MinimumLiquidity: Get<Self::Balance>;
}


#[pallet::pallet]
pub  struct Pallet<T>(_);
```

```
/// 유동성 풀 정보 저장
#[pallet::storage]
#[pallet::getter(fn pools)]
pub type Pools<T: Config> = StorageMap<
_,
Blake2_128Concat,
u32, // Pool ID
PoolInfo<T::AssetId, T::Balance>,
OptionQuery,
>;


/// 다음 풀 ID
#[pallet::storage]
#[pallet::getter(fn next_pool_id)]
pub type NextPoolId<T: Config> = StorageValue<_, u32, ValueQuery>;


/// 자산 쌍별 풀 ID 매핑
#[pallet::storage]
#[pallet::getter(fn asset_pair_pools)]
pub type AssetPairPools<T: Config> = StorageDoubleMap<
_,
Blake2_128Concat,
T::AssetId,
```

```
Blake2_128Concat,

T::AssetId,

u32,

OptionQuery,

>;



/// 유동성 공급자 정보

#[pallet::storage]

#[pallet::getter(fn liquidity_providers)]

pub  type LiquidityProviders<T: Config> = StorageDoubleMap<

_,

Blake2_128Concat,

u32, // Pool ID

Blake2_128Concat,

T::AccountId,

T::Balance, // LP 토큰 수량

ValueQuery,

>;



/// 누적 수수료 (풀별)

#[pallet::storage]

#[pallet::getter(fn accumulated_fees)]

pub  type AccumulatedFees<T: Config> = StorageDoubleMap<
```

```
_,
Blake2_128Concat,
u32, // Pool ID
Blake2_128Concat,
T::AssetId,
T::Balance,
ValueQuery,
>;


/// 팔렛 이벤트
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub  enum Event<T: Config> {
/// 새 유동성 풀이 생성됨 [pool_id, asset_a, asset_b, creator]
PoolCreated {
pool_id: u32,
asset_a: T::AssetId,
asset_b: T::AssetId,
creator: T::AccountId,
},
/// 유동성이 추가됨 [pool_id, provider, amount_a, amount_b, lp_tokens]
LiquidityAdded {
pool_id: u32,
```

```
provider: T::AccountId,

amount_a: T::Balance,

amount_b: T::Balance,

lp_tokens: T::Balance,

},

/// 유동성이 제거됨 [pool_id, provider, amount_a, amount_b, lp_tokens]

LiquidityRemoved {

pool_id: u32,

provider: T::AccountId,

amount_a: T::Balance,

amount_b: T::Balance,

lp_tokens: T::Balance,

},

/// 토큰 스왑이 실행됨 [pool_id, trader, asset_in, asset_out, amount_in, amount_ou

TokenSwapped {

pool_id: u32,

trader: T::AccountId,

asset_in: T::AssetId,

asset_out: T::AssetId,

amount_in: T::Balance,

amount_out: T::Balance,

},

}
```

```
/// 팔렛 오류
#[pallet::error]
pub  enum Error<T> {
/// 풀이 존재하지 않음
PoolNotFound,
/// 풀이 이미 존재함
PoolAlreadyExists,
/// 잔액 부족
InsufficientBalance,
/// 유동성 부족
InsufficientLiquidity,
/// 슬리피지 한계 초과
SlippageExceeded,
/// 잘못된 자산 쌍
InvalidAssetPair,
/// 최소 유동성 미달
InsufficientMinimumLiquidity,
/// 수학적 오버플로우
MathOverflow,
/// 0으로 나누기 시도
DivisionByZero,
/// 잘못된 경로
```

```
InvalidPath,
/// 권한 없음
Unauthorized,
}

/// 팔렛 디스패처블 함수들
#[pallet::call]
impl<T: Config> Pallet<T> {
/// 새로운 유동성 풀 생성
#[pallet::call_index(0)]
#[pallet::weight(10_000)]
pub fn create_pool(
origin: OriginFor<T>,
asset_a: T::AssetId,
asset_b: T::AssetId,
amount_a: T::Balance,
amount_b: T::Balance,
) -> DispatchResult {
let who = ensure_signed(origin)?;

// 자산 쌍 유효성 검사
ensure!(asset_a != asset_b, Error::<T>::InvalidAssetPair);
// 정렬된 자산 쌍 생성
```

```
let (asset_a, asset_b, amount_a, amount_b) = if  asset_a < asset_b {
(asset_a, asset_b, amount_a, amount_b)
} else {
(asset_b, asset_a, amount_b, amount_a)
};

// 풀이 이미 존재하는지 확인
ensure!(
!AssetPairPools::<T>::contains_key(&asset_a, &asset_b),
Error::<T>::PoolAlreadyExists
);

// 최소 유동성 확인
ensure!(
amount_a >= T::MinimumLiquidity::get() && amount_b >= T::MinimumLiquidity::get()
Error::<T>::InsufficientMinimumLiquidity
);

// 사용자 잔액 확인
ensure!(
T::Assets::balance(asset_a, &who) >= amount_a,
Error::<T>::InsufficientBalance
);
```

```
ensure!(
T::Assets::balance(asset_b, &who) >= amount_b,
Error::<T>::InsufficientBalance
);


// 새 풀 ID 생성
let pool_id = Self::next_pool_id();
let next_id = pool_id.checked_add(1).ok_or(Error::<T>::MathOverflow)?;
NextPoolId::<T>::put(next_id);


// 초기 LP 토큰 수량 계산 (기하평균)
let initial_liquidity = Self::calculate_initial_liquidity(amount_a, amount_b)?;


// 풀 정보 생성
let pool_info = PoolInfo {
asset_a,
asset_b,
reserve_a: amount_a,
reserve_b: amount_b,
total_supply: initial_liquidity,
fee_rate: T::DefaultFeeRate::get(),
};
```

```
// 팔렛 계정으로 토큰 전송
let  pallet_account = Self::account_id();
T::Assets::transfer(asset_a, &who, &pallet_account, amount_a, Preservation::Expe
T::Assets::transfer(asset_b, &who, &pallet_account, amount_b, Preservation::Expe

// 스토리지 업데이트
Pools::<T>::insert(&pool_id, &pool_info);
AssetPairPools::<T>::insert(&asset_a, &asset_b, &pool_id);
LiquidityProviders::<T>::insert(&pool_id, &who, initial_liquidity);

// 이벤트 발생
Self::deposit_event(Event::PoolCreated {
pool_id,
asset_a,
asset_b,
creator: who,
});

Ok(())
}

/// 유동성 추가
#[pallet::call_index(1)]
```

```
#[pallet::weight(10_000)]
pub fn add_liquidity(
origin: OriginFor<T>,
pool_id: u32,
amount_a_desired: T::Balance,
amount_b_desired: T::Balance,
amount_a_min: T::Balance,
amount_b_min: T::Balance,
) -> DispatchResult {
let who = ensure_signed(origin)?;


// 풀 존재 확인
let mut pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;


// 최적 비율 계산
let (amount_a, amount_b) = Self::calculate_optimal_amounts(
&pool,
amount_a_desired,
amount_b_desired,
amount_a_min,
amount_b_min,
)?;
```

```
// 사용자 잔액 확인
ensure!(
T::Assets::balance(pool.asset_a, &who) >= amount_a,
Error::<T>::InsufficientBalance
);
ensure!(
T::Assets::balance(pool.asset_b, &who) >= amount_b,
Error::<T>::InsufficientBalance
);

// LP 토큰 수량 계산
let  lp_tokens = Self::calculate_lp_tokens(&pool, amount_a, amount_b)?;

// 팔렛 계정으로 토큰 전송
let  pallet_account = Self::account_id();
T::Assets::transfer(pool.asset_a, &who, &pallet_account, amount_a, Preservation:
T::Assets::transfer(pool.asset_b, &who, &pallet_account, amount_b, Preservation:

// 풀 상태 업데이트
pool.reserve_a = pool.reserve_a.saturating_add(amount_a);
pool.reserve_b = pool.reserve_b.saturating_add(amount_b);
pool.total_supply = pool.total_supply.saturating_add(lp_tokens);
```

```
// 스토리지 업데이트
Pools::<T>::insert(&pool_id, &pool);
LiquidityProviders::<T>::mutate(&pool_id, &who, |balance| {
*balance = balance.saturating_add(lp_tokens);
});

// 이벤트 발생
Self::deposit_event(Event::LiquidityAdded {
pool_id,
provider: who,
amount_a,
amount_b,
lp_tokens,
});

Ok(())
}

/// 유동성 제거
#[pallet::call_index(2)]
#[pallet::weight(10_000)]
pub fn remove_liquidity(
origin: OriginFor<T>,
```

```
pool_id: u32,
lp_tokens: T::Balance,
amount_a_min: T::Balance,
amount_b_min: T::Balance,
) -> DispatchResult {
let  who = ensure_signed(origin)?;

// 풀 존재 확인
let  mut  pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;

// LP 토큰 잔액 확인
let  user_lp_balance = Self::liquidity_providers(pool_id, &who);
ensure!(user_lp_balance >= lp_tokens, Error::<T>::InsufficientBalance);

// 반환할 토큰 수량 계산
let (amount_a, amount_b) = Self::calculate_remove_amounts(&pool, lp_tokens)?;

// 최소 수량 확인
ensure!(amount_a >= amount_a_min, Error::<T>::SlippageExceeded);
ensure!(amount_b >= amount_b_min, Error::<T>::SlippageExceeded);

// 팔렛 계정에서 사용자에게 토큰 전송
let  pallet_account = Self::account_id();
```

```
T::Assets::transfer(pool.asset_a, &pallet_account, &who, amount_a, Preservation:

T::Assets::transfer(pool.asset_b, &pallet_account, &who, amount_b, Preservation:


// 풀 상태 업데이트

pool.reserve_a = pool.reserve_a.saturating_sub(amount_a);

pool.reserve_b = pool.reserve_b.saturating_sub(amount_b);

pool.total_supply = pool.total_supply.saturating_sub(lp_tokens);


// 스토리지 업데이트

Pools::<T>::insert(&pool_id, &pool);

LiquidityProviders::<T>::mutate(&pool_id, &who, |balance| {

*balance = balance.saturating_sub(lp_tokens);

});


// 이벤트 발생

Self::deposit_event(Event::LiquidityRemoved {

pool_id,

provider: who,

amount_a,

amount_b,

lp_tokens,

});
```

```
Ok(())
}

/// 토큰 스왑 (정확한 입력)
#[pallet::call_index(3)]
#[pallet::weight(10_000)]
pub fn swap_exact_tokens_for_tokens(
origin: OriginFor<T>,
amount_in: T::Balance,
amount_out_min: T::Balance,
path: Vec<T::AssetId>,
) -> DispatchResult {
let who = ensure_signed(origin)?;

// 경로 유효성 검사
ensure!(path.len() >= 2, Error::<T>::InvalidPath);

// 스왑 실행
let amounts = Self::execute_swap(&who, amount_in, &path, true)?;
let amount_out = amounts[amounts.len() - 1];

// 슬리피지 확인
ensure!(amount_out >= amount_out_min, Error::<T>::SlippageExceeded);
```

```
Ok(())
}

/// 토큰 스왑 (정확한 출력)
#[pallet::call_index(4)]
#[pallet::weight(10_000)]
pub fn swap_tokens_for_exact_tokens(
origin: OriginFor<T>,
amount_out: T::Balance,
amount_in_max: T::Balance,
path: Vec<T::AssetId>,
) -> DispatchResult {
let who = ensure_signed(origin)?;

// 경로 유효성 검사
ensure!(path.len() >= 2, Error::<T>::InvalidPath);

// 스왑 실행
let amounts = Self::execute_swap(&who, amount_out, &path, false)?;
let amount_in = amounts[0];

// 슬리피지 확인
```

```
ensure!(amount_in <= amount_in_max, Error::<T>::SlippageExceeded);


Ok(())
}
}


/// 헬퍼 함수들
impl<T: Config> Pallet<T> {
/// 팔렛 계정 ID 반환
pub fn account_id() -> T::AccountId {
T::PalletId::get().into_account_truncating()
}


/// 초기 유동성 계산 (기하평균)
fn calculate_initial_liquidity(
amount_a: T::Balance,
amount_b: T::Balance,
) -> Result<T::Balance, Error<T>> {
// sqrt(amount_a * amount_b) - MINIMUM_LIQUIDITY
let product = amount_a.checked_mul(&amount_b).ok_or(Error::<T>::MathOverflow)?;
let sqrt_result = Self::integer_sqrt(product);
let minimum = T::MinimumLiquidity::get();
sqrt_result.checked_sub(&minimum).ok_or(Error::<T>::InsufficientMinimumLiquidity
```

```
}

/// 정수 제곱근 계산 (뉴턴 방법)
fn integer_sqrt(n: T::Balance) -> T::Balance {
if n.is_zero() {
return T::Balance::zero();
}

let mut x = n;
let mut y = (n + T::Balance::from(1u32)) / T::Balance::from(2u32);

while y < x {
x = y;
y = (x + n / x) / T::Balance::from(2u32);
}

x
}

/// 최적 유동성 추가 수량 계산
fn calculate_optimal_amounts(
pool: &PoolInfo<T::AssetId, T::Balance>,
amount_a_desired: T::Balance,
```

```
amount_b_desired: T::Balance,

amount_a_min: T::Balance,

amount_b_min: T::Balance,

) -> Result<(T::Balance, T::Balance), Error<T>> {

// 현재 비율에 맞춰 최적 수량 계산

let  amount_b_optimal = amount_a_desired

.checked_mul(&pool.reserve_b)

.and_then(|x| x.checked_div(&pool.reserve_a))

.ok_or(Error::<T>::MathOverflow)?;



if  amount_b_optimal <= amount_b_desired {

ensure!(amount_b_optimal >= amount_b_min, Error::<T>::SlippageExceeded);

Ok((amount_a_desired, amount_b_optimal))

} else {

let  amount_a_optimal = amount_b_desired

.checked_mul(&pool.reserve_a)

.and_then(|x| x.checked_div(&pool.reserve_b))

.ok_or(Error::<T>::MathOverflow)?;

ensure!(amount_a_optimal >= amount_a_min, Error::<T>::SlippageExceeded);

Ok((amount_a_optimal, amount_b_desired))

}

}
```

```
/// LP 토큰 수량 계산
fn calculate_lp_tokens(
pool: &PoolInfo<T::AssetId, T::Balance>,
amount_a: T::Balance,
amount_b: T::Balance,
) -> Result<T::Balance, Error<T>> {
// min(amount_a * total_supply / reserve_a, amount_b * total_supply / reserve_b)
let lp_a = amount_a
.checked_mul(&pool.total_supply)
.and_then(|x| x.checked_div(&pool.reserve_a))
.ok_or(Error::<T>::MathOverflow)?;

let lp_b = amount_b
.checked_mul(&pool.total_supply)
.and_then(|x| x.checked_div(&pool.reserve_b))
.ok_or(Error::<T>::MathOverflow)?;

Ok(lp_a.min(lp_b))
}

/// 유동성 제거 시 반환 수량 계산
fn calculate_remove_amounts(
pool: &PoolInfo<T::AssetId, T::Balance>,
```

```
lp_tokens: T::Balance,
) -> Result<(T::Balance, T::Balance), Error<T>> {
let amount_a = lp_tokens
.checked_mul(&pool.reserve_a)
.and_then(|x| x.checked_div(&pool.total_supply))
.ok_or(Error::<T>::MathOverflow)?;

let amount_b = lp_tokens
.checked_mul(&pool.reserve_b)
.and_then(|x| x.checked_div(&pool.total_supply))
.ok_or(Error::<T>::MathOverflow)?;

Ok((amount_a, amount_b))
}

/// 스왑 실행
fn execute_swap(
who: &T::AccountId,
amount: T::Balance,
path: &[T::AssetId],
exact_input: bool,
) -> Result<Vec<T::Balance>, Error<T>> {
let mut amounts = if exact_input {
```

```
Self::get_amounts_out(amount, path)?
} else {
Self::get_amounts_in(amount, path)?
};


// 첫 번째 토큰을 팔렛으로 전송
let pallet_account = Self::account_id();
T::Assets::transfer(
path[0],
who,
&pallet_account,
amounts[0],
Preservation::Expendable,
)?;


// 각 스왑 단계 실행
for i in 0..path.len() - 1 {
let (asset_in, asset_out) = (path[i], path[i + 1]);
let (amount_in, amount_out) = (amounts[i], amounts[i + 1]);


// 풀 찾기
let pool_id = Self::find_pool(asset_in, asset_out)?;
let mut pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
```

```
// 스왑 실행 및 풀 상태 업데이트
Self::execute_single_swap(&mut  pool, asset_in, asset_out, amount_in, amount_out
Pools::<T>::insert(&pool_id, &pool);


// 마지막 단계가 아니면 다음 스왑을 위해 토큰을 팔렛에 보관
// 마지막 단계면 사용자에게 직접 전송
let  recipient = if  i == path.len() - 2 { who } else { &pallet_account };
T::Assets::transfer(
asset_out,
&pallet_account,
recipient,
amount_out,
Preservation::Expendable,
)?;


// 이벤트 발생
Self::deposit_event(Event::TokenSwapped {
pool_id,
trader: who.clone(),
asset_in,
asset_out,
amount_in,
```

```
amount_out,
});
}

Ok(amounts)
}

/// 출력 수량 계산 (정확한 입력)
fn get_amounts_out(
amount_in: T::Balance,
path: &[T::AssetId],
) -> Result<Vec<T::Balance>, Error<T>> {
let mut amounts = vec![amount_in];

for i in 0..path.len() - 1 {
let pool_id = Self::find_pool(path[i], path[i + 1])?;
let pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
let amount_out = Self::get_amount_out(
amounts[i],
path[i],
&pool,
)?;
amounts.push(amount_out);
```

```
}

Ok(amounts)
}

/// 입력 수량 계산 (정확한 출력)
fn get_amounts_in(
amount_out: T::Balance,
path: &[T::AssetId],
) -> Result<Vec<T::Balance>, Error<T>> {
let mut amounts = vec![T::Balance::zero(); path.len()];
amounts[path.len() - 1] = amount_out;

for i in (0..path.len() - 1).rev() {
let pool_id = Self::find_pool(path[i], path[i + 1])?;
let pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
let amount_in = Self::get_amount_in(
amounts[i + 1],
path[i],
&pool,
)?;
amounts[i] = amount_in;
}
```

```
Ok(amounts)
}

/// 단일 스왑의 출력 수량 계산 (AMM 공식)
fn  get_amount_out(
amount_in: T::Balance,
asset_in: T::AssetId,
pool: &PoolInfo<T::AssetId, T::Balance>,
) -> Result<T::Balance, Error<T>> {
let (reserve_in, reserve_out) = if  asset_in == pool.asset_a {
(pool.reserve_a, pool.reserve_b)
} else {
(pool.reserve_b, pool.reserve_a)
};

// 수수료 적용된 입력 수량
let  fee_rate = pool.fee_rate;
let  amount_in_with_fee = amount_in
.checked_mul(&(10000u32.saturating_sub(fee_rate)).into())
.ok_or(Error::<T>::MathOverflow)?;

// AMM 공식: amount_out = (amount_in_with_fee * reserve_out) / (reserve_in * 1000
```

```
let  numerator = amount_in_with_fee
.checked_mul(&reserve_out)
.ok_or(Error::<T>::MathOverflow)?;


let  denominator = reserve_in
.checked_mul(&10000u32.into())
.and_then(|x| x.checked_add(&amount_in_with_fee))
.ok_or(Error::<T>::MathOverflow)?;


numerator.checked_div(&denominator).ok_or(Error::<T>::DivisionByZero)
}


/// 단일 스왑의 입력 수량 계산
fn  get_amount_in(
amount_out: T::Balance,
asset_in: T::AssetId,
pool: &PoolInfo<T::AssetId, T::Balance>,
) -> Result<T::Balance, Error<T>> {
let (reserve_in, reserve_out) = if  asset_in == pool.asset_a {
(pool.reserve_a, pool.reserve_b)
} else {
(pool.reserve_b, pool.reserve_a)
};
```

```
// AMM 공식 역계산
let fee_rate = pool.fee_rate;
let numerator = reserve_in
.checked_mul(&amount_out)
.and_then(|x| x.checked_mul(&10000u32.into()))
.ok_or(Error::<T>::MathOverflow)?;

let denominator = reserve_out
.checked_sub(&amount_out)
.and_then(|x| x.checked_mul(&(10000u32.saturating_sub(fee_rate)).into()))
.ok_or(Error::<T>::MathOverflow)?;

numerator.checked_div(&denominator)
.and_then(|x| x.checked_add(&T::Balance::from(1u32)))
.ok_or(Error::<T>::DivisionByZero)
}

/// 자산 쌍에 해당하는 풀 찾기
fn find_pool(asset_a: T::AssetId, asset_b: T::AssetId) -> Result<u32, Error<T>>
let (asset_a, asset_b) = if asset_a < asset_b {
(asset_a, asset_b)
} else {
```

```
(asset_b, asset_a)
};

Self::asset_pair_pools(asset_a, asset_b).ok_or(Error::<T>::PoolNotFound)
}

/// 단일 스왑 실행 및 풀 상태 업데이트
fn execute_single_swap(
pool: &mut PoolInfo<T::AssetId, T::Balance>,
asset_in: T::AssetId,
asset_out: T::AssetId,
amount_in: T::Balance,
amount_out: T::Balance,
) -> Result<(), Error<T>> {
if asset_in == pool.asset_a {
pool.reserve_a = pool.reserve_a.saturating_add(amount_in);
pool.reserve_b = pool.reserve_b.saturating_sub(amount_out);
} else {
pool.reserve_b = pool.reserve_b.saturating_add(amount_in);
pool.reserve_a = pool.reserve_a.saturating_sub(amount_out);
}

Ok(())
```

```
}

/// 풀의 현재 가격 조회
pub fn get_price(
pool_id: u32,
asset: T::AssetId,
) -> Result<T::Balance, Error<T>> {
let pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
if asset == pool.asset_a {
// asset_a의 가격 = reserve_b / reserve_a
pool.reserve_b.checked_div(&pool.reserve_a).ok_or(Error::<T>::DivisionByZero)
} else if asset == pool.asset_b {
// asset_b의 가격 = reserve_a / reserve_b
pool.reserve_a.checked_div(&pool.reserve_b).ok_or(Error::<T>::DivisionByZero)
} else {
Err(Error::<T>::InvalidAssetPair)
}
}
}
}

// 데이터 구조 정의
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
```

```
pub struct PoolInfo<AssetId, Balance> {
pub asset_a: AssetId,
pub asset_b: AssetId,
pub reserve_a: Balance,
pub reserve_b: Balance,
pub total_supply: Balance,
pub fee_rate: u32,
}
```

이제 첫 번째 고급 튜토리얼의 기본 구조를 완성했습니다. 계속해서 나머지 부분을 작성하겠습니다.

<function_calls>

append

3단계: 런타임 통합

3.1 런타임 Cargo.toml 수정

```
# runtime/Cargo.toml의 [dependencies] 섹션에 추가
pallet-defi-dex = { version = "4.0.0-dev", default-features = false, path = "../
pallet-assets = { version = "4.0.0-dev", default-features = false, git = "https:

# [features] 섹션의 std 배열에 추가
std = [
# ... 기존 항목들
```

```
"pallet-defi-dex/std",
"pallet-assets/std",
]
```

3.2 런타임 설정

```
// runtime/src/lib.rs에 추가

use frame_support::PalletId;

// Assets 팔렛 설정
impl pallet_assets::Config for Runtime {
type RuntimeEvent = RuntimeEvent;
type Balance = Balance;
type AssetId = u32;
type AssetIdParameter = codec::Compact<u32>;
type Currency = Balances;
type CreateOrigin = AsEnsureOriginWithArg<EnsureSigned<AccountId>>;
type ForceOrigin = EnsureRoot<AccountId>;
type AssetDeposit = ConstU128<100>;
type AssetAccountDeposit = ConstU128<1>;
type MetadataDepositBase = ConstU128<10>;
type MetadataDepositPerByte = ConstU128<1>;
```

```
type ApprovalDeposit = ExistentialDeposit;

type StringLimit = ConstU32<50>;

type Freezer = ();

type Extra = ();

type CallbackHandle = ();

type WeightInfo = pallet_assets::weights::SubstrateWeight<Runtime>;

type RemoveItemsLimit = ConstU32<1000>;

}



// DeFi DEX 팔렛 설정

parameter_types! {

pub  const DexPalletId: PalletId = PalletId(*b"py/defid");

pub  const DefaultFeeRate: u32 = 30; // 0.3%

pub  const MinimumLiquidity: Balance = 1000;

}



impl pallet_defi_dex::Config for Runtime {

type RuntimeEvent = RuntimeEvent;

type AssetId = u32;

type Balance = Balance;

type Assets = Assets;

type PalletId = DexPalletId;

type DefaultFeeRate = DefaultFeeRate;
```

```
type MinimumLiquidity = MinimumLiquidity;

}


// construct_runtime! 매크로에 팔렛 추가

construct_runtime!(

pub  struct Runtime {

// ... 기존 팔렛들

Assets: pallet_assets,

DefiDex: pallet_defi_dex,

}

);
```

4단계: 테스트 코드 작성

4.1 Mock 런타임 생성

```
// pallets/defi-dex/src/mock.rs

use  crate  as pallet_defi_dex;

use frame_support::{

derive_impl,

parameter_types,

traits::{AsEnsureOriginWithArg, ConstU16, ConstU32, ConstU64, ConstU128},

PalletId,
```

```
};
use sp_core::H256;
use sp_runtime::{
traits::{BlakeTwo256, IdentityLookup},
BuildStorage,
};


type Block = frame_system::mocking::MockBlock<Test>;


frame_support::construct_runtime!(
pub  enum Test {
System: frame_system,
Balances: pallet_balances,
Assets: pallet_assets,
DefiDex: pallet_defi_dex,
}
);


#[derive_impl(frame_system::config_preludes::TestDefaultConfig as frame_system::
impl frame_system::Config for Test {
type BaseCallFilter = frame_support::traits::Everything;
type BlockWeights = ();
type BlockLength = ();
```

```
type DbWeight = ();

type RuntimeOrigin = RuntimeOrigin;

type RuntimeCall = RuntimeCall;

type Nonce = u64;

type Hash = H256;

type Hashing = BlakeTwo256;

type AccountId = u64;

type Lookup = IdentityLookup<Self::AccountId>;

type Block = Block;

type RuntimeEvent = RuntimeEvent;

type BlockHashCount = ConstU64<250>;

type Version = ();

type PalletInfo = PalletInfo;

type AccountData = pallet_balances::AccountData<u128>;

type OnNewAccount = ();

type OnKilledAccount = ();

type SystemWeightInfo = ();

type SS58Prefix = ConstU16<42>;

type OnSetCode = ();

type MaxConsumers = frame_support::traits::ConstU32<16>;

}


impl pallet_balances::Config for Test {
```

```
type MaxLocks = ConstU32<50>;

type MaxReserves = ();

type ReserveIdentifier = [u8; 8];

type Balance = u128;

type RuntimeEvent = RuntimeEvent;

type DustRemoval = ();

type ExistentialDeposit = ConstU128<500>;

type AccountStore = System;

type WeightInfo = pallet_balances::weights::SubstrateWeight<Test>;

type FreezeIdentifier = ();

type MaxFreezes = ();

type RuntimeHoldReason = ();

type RuntimeFreezeReason = ();

}


impl pallet_assets::Config for Test {

type RuntimeEvent = RuntimeEvent;

type Balance = u128;

type AssetId = u32;

type AssetIdParameter = codec::Compact<u32>;

type Currency = Balances;

type CreateOrigin = AsEnsureOriginWithArg<frame_system::EnsureSigned<u64>>;

type ForceOrigin = frame_system::EnsureRoot<u64>;
```

```
type AssetDeposit = ConstU128<100>;

type AssetAccountDeposit = ConstU128<1>;

type MetadataDepositBase = ConstU128<10>;

type MetadataDepositPerByte = ConstU128<1>;

type ApprovalDeposit = ConstU128<1>;

type StringLimit = ConstU32<50>;

type Freezer = ();

type Extra = ();

type CallbackHandle = ();

type WeightInfo = pallet_assets::weights::SubstrateWeight<Test>;

type RemoveItemsLimit = ConstU32<1000>;

}


parameter_types! {

pub  const DexPalletId: PalletId = PalletId(*b"py/defid");

pub  const DefaultFeeRate: u32 = 30; // 0.3%

pub  const MinimumLiquidity: u128 = 1000;

}


impl pallet_defi_dex::Config for Test {

type RuntimeEvent = RuntimeEvent;

type AssetId = u32;

type Balance = u128;
```

```
type Assets = Assets;

type PalletId = DexPalletId;

type DefaultFeeRate = DefaultFeeRate;

type MinimumLiquidity = MinimumLiquidity;

}



pub fn new_test_ext() -> sp_io::TestExternalities {

let mut t = frame_system::GenesisConfig::<Test>::default().build_storage().unw



pallet_balances::GenesisConfig::<Test> {

balances: vec![

(1, 1_000_000_000_000),

(2, 1_000_000_000_000),

(3, 1_000_000_000_000),

],

}

.assimilate_storage(&mut t)

.unwrap();



let mut ext = sp_io::TestExternalities::new(t);

ext.execute_with(|| System::set_block_number(1));

ext
```

```
}
```

4.2 단위 테스트 작성

```
// pallets/defi-dex/src/tests.rs
use super::*;
use crate::{mock::*, Error, Event};
use frame_support::{assert_noop, assert_ok, traits::Get};

#[test]
fn create_pool_works() {
new_test_ext().execute_with(|| {
// 테스트 자산 생성
assert_ok!(Assets::create(RuntimeOrigin::signed(1), 0.into(), 1, 1));
assert_ok!(Assets::create(RuntimeOrigin::signed(1), 1.into(), 1, 1));

// 자산 발행
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 0.into(), 1, 1_000_000));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 1, 1_000_000));

// 풀 생성
assert_ok!(DefiDex::create_pool(
RuntimeOrigin::signed(1),
```

```
0,
1,
100_000,
200_000
));

// 풀 정보 확인
let  pool = DefiDex::pools(0).unwrap();
assert_eq!(pool.asset_a, 0);
assert_eq!(pool.asset_b, 1);
assert_eq!(pool.reserve_a, 100_000);
assert_eq!(pool.reserve_b, 200_000);

// 이벤트 확인
System::assert_last_event(Event::PoolCreated {
pool_id: 0,
asset_a: 0,
asset_b: 1,
creator: 1,
}.into());
});
}
```

```
#[test]
fn  create_pool_fails_with_same_assets() {
new_test_ext().execute_with(|| {
assert_ok!(Assets::create(RuntimeOrigin::signed(1), 0.into(), 1, 1));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 0.into(), 1, 1_000_000));

// 같은 자산으로 풀 생성 시도
assert_noop!(
DefiDex::create_pool(RuntimeOrigin::signed(1), 0, 0, 100_000, 200_000),
Error::<Test>::InvalidAssetPair
);
});
}

#[test]
fn  add_liquidity_works() {
new_test_ext().execute_with(|| {
// 풀 생성
setup_pool();

// 유동성 추가
assert_ok!(DefiDex::add_liquidity(
RuntimeOrigin::signed(2),
```

```
0,
50_000,
100_000,
40_000,
80_000
));

// 풀 상태 확인
let pool = DefiDex::pools(0).unwrap();
assert_eq!(pool.reserve_a, 150_000);
assert_eq!(pool.reserve_b, 300_000);

// LP 토큰 확인
let lp_balance = DefiDex::liquidity_providers(0, 2);
assert!(lp_balance > 0);
});
}

#[test]
fn swap_exact_tokens_for_tokens_works() {
new_test_ext().execute_with(|| {
// 풀 생성 및 유동성 추가
setup_pool_with_liquidity();
```

```
let initial_balance_0 = Assets::balance(0, &3);
let initial_balance_1 = Assets::balance(1, &3);

// 스왑 실행
assert_ok!(DefiDex::swap_exact_tokens_for_tokens(
RuntimeOrigin::signed(3),
10_000,
0,
vec![0, 1]
));

// 잔액 변화 확인
let final_balance_0 = Assets::balance(0, &3);
let final_balance_1 = Assets::balance(1, &3);

assert_eq!(final_balance_0, initial_balance_0 - 10_000);
assert!(final_balance_1 > initial_balance_1);
});
}

#[test]
fn remove_liquidity_works() {
```

```
new_test_ext().execute_with(|| {
setup_pool_with_liquidity();

let lp_balance = DefiDex::liquidity_providers(0, 1);
let remove_amount = lp_balance / 2;

let initial_balance_0 = Assets::balance(0, &1);
let initial_balance_1 = Assets::balance(1, &1);

// 유동성 제거
assert_ok!(DefiDex::remove_liquidity(
RuntimeOrigin::signed(1),
0,
remove_amount,
0,
0
));

// 잔액 증가 확인
let final_balance_0 = Assets::balance(0, &1);
let final_balance_1 = Assets::balance(1, &1);

assert!(final_balance_0 > initial_balance_0);
```

```
assert!(final_balance_1 > initial_balance_1);
});
}

#[test]
fn  slippage_protection_works() {
new_test_ext().execute_with(|| {
setup_pool_with_liquidity();

// 높은 최소 출력으로 스왑 시도 (실패해야 함)
assert_noop!(
DefiDex::swap_exact_tokens_for_tokens(
RuntimeOrigin::signed(3),
10_000,
50_000, // 너무 높은 최소 출력
vec![0, 1]
),
Error::<Test>::SlippageExceeded
);
});
}

#[test]
```

```rust
fn price_calculation_works() {
new_test_ext().execute_with(|| {
setup_pool();

// 가격 조회
let price_0 = DefiDex::get_price(0, 0).unwrap();
let price_1 = DefiDex::get_price(0, 1).unwrap();

// 초기 비율: 100_000:200_000 = 1:2
assert_eq!(price_0, 2); // asset_0의 가격 = reserve_1 / reserve_0
assert_eq!(price_1, 0); // asset_1의 가격 = reserve_0 / reserve_1 (정수 나눗셈으로
});
}

// 헬퍼 함수들
fn setup_pool() {
// 자산 생성
assert_ok!(Assets::create(RuntimeOrigin::signed(1), 0.into(), 1, 1));
assert_ok!(Assets::create(RuntimeOrigin::signed(1), 1.into(), 1, 1));

// 자산 발행
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 0.into(), 1, 1_000_000));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 1, 1_000_000));
```

```
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 0.into(), 2, 1_000_000));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 2, 1_000_000));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 0.into(), 3, 1_000_000));
assert_ok!(Assets::mint(RuntimeOrigin::signed(1), 1.into(), 3, 1_000_000));

// 풀 생성
assert_ok!(DefiDex::create_pool(
RuntimeOrigin::signed(1),
0,
1,
100_000,
200_000
));
}

fn  setup_pool_with_liquidity() {
setup_pool();

// 추가 유동성 제공
assert_ok!(DefiDex::add_liquidity(
RuntimeOrigin::signed(2),
0,
100_000,
```

```
200_000,
90_000,
180_000
));
}
```

5단계: 고급 기능 구현

5.1 다중 홉 스왑 최적화

```
// pallets/defi-dex/src/lib.rs에 추가

impl<T: Config> Pallet<T> {
/// 최적 스왑 경로 찾기
pub fn find_best_path(
asset_in: T::AssetId,
asset_out: T::AssetId,
amount_in: T::Balance,
) -> Result<Vec<T::AssetId>, Error<T>> {
// 직접 경로 확인
if Self::find_pool(asset_in, asset_out).is_ok() {
return Ok(vec![asset_in, asset_out]);
}
```

```
// 간접 경로 탐색 (최대 3홉)
let mut best_path = Vec::new();
let mut best_amount_out = T::Balance::zero();


// 모든 풀을 순회하며 중간 자산 찾기
for (pool_id, pool) in Pools::<T>::iter() {
// asset_in -> intermediate -> asset_out 경로 확인
let intermediate_assets = vec![pool.asset_a, pool.asset_b];
for intermediate in intermediate_assets {
if intermediate != asset_in && intermediate != asset_out {
let path = vec![asset_in, intermediate, asset_out];
if let Ok(amounts) = Self::get_amounts_out(amount_in, &path) {
let amount_out = amounts[amounts.len() - 1];
if amount_out > best_amount_out {
best_amount_out = amount_out;
best_path = path;
}
}
}
}
}
```

```
if best_path.is_empty() {
Err(Error::<T>::InvalidPath)
} else {
Ok(best_path)
}
}

/// 가격 영향도 계산
pub fn calculate_price_impact(
pool_id: u32,
asset_in: T::AssetId,
amount_in: T::Balance,
) -> Result<Permill, Error<T>> {
let pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
let (reserve_in, reserve_out) = if asset_in == pool.asset_a {
(pool.reserve_a, pool.reserve_b)
} else {
(pool.reserve_b, pool.reserve_a)
};

// 현재 가격
let current_price = reserve_out.checked_div(&reserve_in)
.ok_or(Error::<T>::DivisionByZero)?;
```

```
// 스왑 후 예상 가격
let amount_out = Self::get_amount_out(amount_in, asset_in, &pool)?;
let new_reserve_in = reserve_in.saturating_add(amount_in);
let new_reserve_out = reserve_out.saturating_sub(amount_out);
let new_price = new_reserve_out.checked_div(&new_reserve_in)
.ok_or(Error::<T>::DivisionByZero)?;

// 가격 영향도 = |new_price - current_price| / current_price
let price_diff = if new_price > current_price {
new_price.saturating_sub(current_price)
} else {
current_price.saturating_sub(new_price)
};

let impact = price_diff.checked_mul(&1_000_000u32.into())
.and_then(|x| x.checked_div(&current_price))
.ok_or(Error::<T>::MathOverflow)?;

Ok(Permill::from_parts(impact.try_into().unwrap_or(1_000_000)))
}
}
```

5.2 수수료 분배 시스템

```
// 수수료 분배 함수 추가
impl<T: Config> Pallet<T> {
/// 수수료 분배
#[pallet::call_index(5)]
#[pallet:weight(10_000)]
pub fn distribute_fees(
origin: OriginFor<T>,
pool_id: u32,
) -> DispatchResult {
let _who = ensure_signed(origin)?;

let pool = Self::pools(pool_id).ok_or(Error::<T>::PoolNotFound)?;
// 누적된 수수료 확인
let fee_a = Self::accumulated_fees(pool_id, pool.asset_a);
let fee_b = Self::accumulated_fees(pool_id, pool.asset_b);

if fee_a.is_zero() && fee_b.is_zero() {
return Ok(());
}

// 모든 LP 토큰 보유자에게 비례 분배
```

```
let  total_supply = pool.total_supply;

for (account, lp_balance) in LiquidityProviders::<T>::iter_prefix(pool_id) {

if  lp_balance.is_zero() {

continue;

}



// 비례 계산

let  share_a = fee_a.checked_mul(&lp_balance)

.and_then(|x| x.checked_div(&total_supply))

.unwrap_or_default();

let  share_b = fee_b.checked_mul(&lp_balance)

.and_then(|x| x.checked_div(&total_supply))

.unwrap_or_default();



// 수수료 지급

if !share_a.is_zero() {

let  pallet_account = Self::account_id();

T::Assets::transfer(

pool.asset_a,

&pallet_account,

&account,

share_a,

Preservation::Expendable,
```

```
)?;
}

if !share_b.is_zero() {
let  pallet_account = Self::account_id();
T::Assets::transfer(
pool.asset_b,
&pallet_account,
&account,
share_b,
Preservation::Expendable,
)?;
}
}

// 누적 수수료 초기화
AccumulatedFees::<T>::remove(pool_id, pool.asset_a);
AccumulatedFees::<T>::remove(pool_id, pool.asset_b);

Ok(())
}

/// 스왑 시 수수료 누적
```

```
fn accumulate_fees(
pool_id: u32,
asset: T::AssetId,
fee_amount: T::Balance,
) {
AccumulatedFees::<T>::mutate(pool_id, asset, |accumulated| {
*accumulated = accumulated.saturating_add(fee_amount);
});
}
}
```

6단계: 프론트엔드 통합 예제

6.1 Polkadot.js API 사용 예제

```
// frontend/dex-interface.js
import { ApiPromise, WsProvider } from '@polkadot/api';
import { Keyring } from '@polkadot/keyring';

class DexInterface {
constructor(wsEndpoint = 'ws://localhost:9944') {
this.wsEndpoint = wsEndpoint;
this.api = null;
```

```
this.keyring = new Keyring({ type: 'sr25519' });
}

async connect() {
const wsProvider = new WsProvider(this.wsEndpoint);
this.api = await ApiPromise.create({ provider: wsProvider });
console.log('Connected to Substrate node');
}

async createPool(signer, assetA, assetB, amountA, amountB) {
const tx = this.api.tx.defiDex.createPool(assetA, assetB, amountA, amountB
return new Promise((resolve, reject) => {
tx.signAndSend(signer, ({ status, events }) => {
if (status.isInBlock) {
console.log(`Transaction included in block ${status.asInBlock}`);
// 이벤트 확인
events.forEach(({ event }) => {
if (event.section === 'defiDex' && event.method === 'PoolCreated') {
const [poolId, assetA, assetB, creator] = event.data;
resolve({
poolId: poolId.toNumber(),
assetA: assetA.toNumber(),
assetB: assetB.toNumber(),
```

```
creator: creator.toString()
});
}
});
}  else  if (status.isError) {
reject(new  Error('Transaction failed'));
}
});
});
}

async  addLiquidity(signer,  poolId,  amountADesired,  amountBDesired,  amountAM
const  tx  =  this.api.tx.defiDex.addLiquidity(
poolId,
amountADesired,
amountBDesired,
amountAMin,
amountBMin
);

return  this.sendTransaction(tx,  signer,  'LiquidityAdded');
}
```

```
async swapExactTokensForTokens(signer, amountIn, amountOutMin, path) {
const tx = this.api.tx.defiDex.swapExactTokensForTokens(
amountIn,
amountOutMin,
path
);

return this.sendTransaction(tx, signer, 'TokenSwapped');
}

async getPoolInfo(poolId) {
const pool = await this.api.query.defiDex.pools(poolId);
if (pool.isSome) {
const poolData = pool.unwrap();
return {
assetA: poolData.asset_a.toNumber(),
assetB: poolData.asset_b.toNumber(),
reserveA: poolData.reserve_a.toString(),
reserveB: poolData.reserve_b.toString(),
totalSupply: poolData.total_supply.toString(),
feeRate: poolData.fee_rate.toNumber()
};
}
```

```
return null;
}

async getAmountOut(amountIn, path) {
// 체인 상태를 직접 조회하여 계산
const amounts = [];
amounts.push(amountIn);

for (let i = 0; i < path.length - 1; i++) {
const assetIn = path[i];
const assetOut = path[i + 1];
// 풀 찾기
const poolId = await this.findPool(assetIn, assetOut);
if (poolId === null) {
throw new Error(`Pool not found for ${assetIn}-${assetOut}`);
}

const pool = await this.getPoolInfo(poolId);
const amountOut = this.calculateAmountOut(amounts[i], assetIn, pool);
amounts.push(amountOut);
}

return amounts;
```

```
}

calculateAmountOut(amountIn, assetIn, pool) {
const reserveIn = assetIn === pool.assetA ?
BigInt(pool.reserveA) : BigInt(pool.reserveB);
const reserveOut = assetIn === pool.assetA ?
BigInt(pool.reserveB) : BigInt(pool.reserveA);

const feeRate = BigInt(pool.feeRate);
const amountInWithFee = BigInt(amountIn) * (10000n - feeRate);
const numerator = amountInWithFee * reserveOut;
const denominator = reserveIn * 10000n + amountInWithFee;
return (numerator / denominator).toString();
}

async findPool(assetA, assetB) {
// 정렬된 자산 쌍으로 풀 ID 조회
const [sortedA, sortedB] = assetA < assetB ? [assetA, assetB] : [assetB,
const poolId = await this.api.query.defiDex.assetPairPools(sortedA, sortedB
return poolId.isSome ? poolId.unwrap().toNumber() : null;
}

async sendTransaction(tx, signer, eventMethod) {
```

```
return new Promise((resolve, reject) => {
tx.signAndSend(signer, ({ status, events }) => {
if (status.isInBlock) {
events.forEach(({ event }) => {
if (event.section === 'defiDex' && event.method === eventMethod) {
resolve(event.data);
}
});
} else if (status.isError) {
reject(new Error('Transaction failed'));
}
});
});
}
}

// 사용 예제
async function example() {
const dex = new DexInterface();
await dex.connect();

// 계정 설정
const alice = dex.keyring.addFromUri('//Alice');
```

```
try {
// 풀 생성
const poolResult = await dex.createPool(alice, 0, 1, 100000, 200000);
console.log('Pool created:', poolResult);

// 유동성 추가
await dex.addLiquidity(alice, poolResult.poolId, 50000, 100000, 45000, 90000);
console.log('Liquidity added');

// 스왑 실행
const swapResult = await dex.swapExactTokensForTokens(alice, 10000, 0, [0,
console.log('Swap completed:', swapResult);

// 풀 정보 조회
const poolInfo = await dex.getPoolInfo(poolResult.poolId);
console.log('Pool info:', poolInfo);

} catch (error) {
console.error('Error:', error);
}
}
```

6.2 React 컴포넌트 예제

```
// frontend/components/SwapInterface.jsx
import React, { useState, useEffect } from 'react';
import { DexInterface } from '../dex-interface';

const SwapInterface = ({ api, account }) => {
const [dex, setDex] = useState(null);
const [tokenIn, setTokenIn] = useState('');
const [tokenOut, setTokenOut] = useState('');
const [amountIn, setAmountIn] = useState('');
const [amountOut, setAmountOut] = useState('');
const [slippage, setSlippage] = useState(0.5);
const [loading, setLoading] = useState(false);

useEffect(() => {
if (api) {
const dexInterface = new DexInterface();
dexInterface.api = api;
setDex(dexInterface);
}
}, [api]);
```

```
const calculateAmountOut = async () => {
if (!dex || !amountIn || !tokenIn || !tokenOut) return;

try {
const path = [parseInt(tokenIn), parseInt(tokenOut)];
const amounts = await dex.getAmountOut(amountIn, path);
setAmountOut(amounts[amounts.length - 1]);
} catch (error) {
console.error('Error calculating amount out:', error);
}
};

const executeSwap = async () => {
if (!dex || !account || !amountIn || !amountOut) return;

setLoading(true);
try {
const path = [parseInt(tokenIn), parseInt(tokenOut)];
const minAmountOut = Math.floor(amountOut * (1 - slippage / 100));
await dex.swapExactTokensForTokens(
account,
amountIn,
minAmountOut.toString(),
```

```
path

);

alert('Swap completed successfully!');

setAmountIn('');

setAmountOut('');

}  catch (error) {

console.error('Swap failed:',  error);

alert('Swap failed: '  +  error.message);

}  finally  {

setLoading(false);

}

};



return (

<div  className="swap-interface">

<h2>Token Swap</h2>

<div  className="swap-form">

<div  className="input-group">

<label>From Token ID:</label>

<input

type="number"

value={tokenIn}

onChange={(e)  =>  setTokenIn(e.target.value)}
```

```
placeholder="Enter token ID"
/>
</div>


<div  className="input-group">
<label>Amount In:</label>
<input
type="number"
value={amountIn}
onChange={(e)  =>  {
setAmountIn(e.target.value);
calculateAmountOut();
}}
placeholder="Enter amount"
/>
</div>


<div  className="swap-arrow">↓</div>


<div  className="input-group">
<label>To Token ID:</label>
<input
type="number"
```

```
value={tokenOut}
onChange={(e) => setTokenOut(e.target.value)}
placeholder="Enter token ID"
/>
</div>

<div className="input-group">
<label>Amount Out (estimated):</label>
<input
type="text"
value={amountOut}
readOnly
placeholder="Calculated amount"
/>
</div>

<div className="input-group">
<label>Slippage Tolerance (%):</label>
<input
type="number"
step="0.1"
value={slippage}
onChange={(e) => setSlippage(parseFloat(e.target.value))}
```

```
/>
</div>

<button
onClick={executeSwap}
disabled={loading || !amountIn || !amountOut}
className="swap-button"
>
{loading ? 'Swapping...' : 'Swap Tokens'}
</button>
</div>
</div>
);
};

export default SwapInterface;
```

7단계: 보안 고려사항 및 최적화

7.1 보안 체크리스트

```
// 보안 강화 함수들
impl<T: Config> Pallet<T> {
```

```
/// 재진입 공격 방지
fn ensure_no_reentrancy() -> DispatchResult {
// 실제 구현에서는 ReentrancyGuard 패턴 사용
Ok(())
}

/// 플래시론 공격 방지
fn validate_swap_amounts(
pool: &PoolInfo<T::AssetId, T::Balance>,
amount_in: T::Balance,
amount_out: T::Balance,
) -> DispatchResult {
// K 불변성 확인 (x * y >= k)
let new_reserve_a = if pool.asset_a == asset_in {
pool.reserve_a.saturating_add(amount_in)
} else {
pool.reserve_a.saturating_sub(amount_out)
};

let new_reserve_b = if pool.asset_b == asset_in {
pool.reserve_b.saturating_add(amount_in)
} else {
pool.reserve_b.saturating_sub(amount_out)
```

```
};

let old_k = pool.reserve_a.saturating_mul(pool.reserve_b);
let new_k = new_reserve_a.saturating_mul(new_reserve_b);

ensure!(new_k >= old_k, Error::<T>::InvalidSwap);
Ok(())
}

/// 대량 거래 제한
fn check_trade_limits(
pool: &PoolInfo<T::AssetId, T::Balance>,
amount: T::Balance,
) -> DispatchResult {
let max_trade_ratio = Permill::from_percent(10); // 10% 제한
let reserve = pool.reserve_a.max(pool.reserve_b);
let max_amount = max_trade_ratio * reserve;

ensure!(amount <= max_amount, Error::<T>::TradeTooLarge);
Ok(())
}
}
```

7.2 가스 최적화

```
// 가중치 최적화
#[pallet::weight(T::DbWeight::get().reads_writes(3, 2))]
pub fn optimized_swap(
origin: OriginFor<T>,
// ... 파라미터들
) -> DispatchResult {
// 최적화된 구현
}

// 배치 처리
#[pallet::call_index(6)]
#[pallet::weight(10_000 * operations.len() as u64)]
pub fn batch_operations(
origin: OriginFor<T>,
operations: Vec<DexOperation<T::AssetId, T::Balance>>,
) -> DispatchResult {
let who = ensure_signed(origin)?;

for operation in operations {
match operation {
DexOperation::Swap { amount_in, path, min_amount_out } => {
```

```
Self::execute_swap(&who, amount_in, &path, true)?;

},

DexOperation::AddLiquidity { pool_id, amounts } => {

// 유동성 추가 로직

},

// 기타 연산들...

}

}


Ok(())

}
```

8단계: 실제 테스트 및 배포

8.1 통합 테스트

```
# 전체 테스트 실행

cargo test -p pallet-defi-dex


# 벤치마크 실행

cargo test -p pallet-defi-dex --features runtime-benchmarks


# 런타임 빌드 테스트
```

```
cargo build --release
```

8.2 실제 사용 시나리오 테스트

```
# 노드 시작
./target/release/node-template --dev --tmp

# Polkadot.js Apps에서 테스트:
# 1. Assets 팔렛으로 테스트 토큰 생성
# 2. DeFi DEX 팔렛으로 풀 생성
# 3. 유동성 추가/제거 테스트
# 4. 다양한 스왑 시나리오 테스트
```

요약

이 고급 튜토리얼에서는 다음을 구현했습니다:

1. **완전한 AMM DEX**: 유동성 풀, 토큰 스왑, LP 토큰 시스템
2. **고급 수학**: 제곱근 계산, 슬리피지 보호, 가격 영향도 계산
3. **보안 기능**: 재진입 방지, K 불변성 검증, 거래 한도
4. **최적화**: 가중치 최적화, 배치 처리, 다중 홉 라우팅
5. **프론트엔드 통합**: JavaScript API, React 컴포넌트
6. **포괄적 테스트**: 단위 테스트, 통합 테스트, 시나리오 테스트

다음 튜토리얼에서는 XCM을 사용한 크로스체인 자산 전송을 구현하겠습니다.

참고 자료

- Uniswap V2 백서
- AMM 수학 공식
- Substrate Assets 팔렛
- DeFi 보안 베스트 프랙티스